A Gentle Introduction to SARIMA for Time Series Forecasting in Python

by Jason Brownlee on August 21, 2019 in Time Series 137

Autoregressive Integrated Moving Average, or ARIMA, is one of the most widely used forecasting methods for univariate time series data forecasting.

Although the method can handle data with a trend, it does not support time series with a seasonal component.

An extension to ARIMA that supports the direct modeling of the seasonal component of the series is called SARIMA.

In this tutorial, you will discover the Seasonal Autoregressive Integrated Moving Average, or SARIMA, method for time series forecasting with univariate data containing trends and seasonality.

After completing this tutorial, you will know:

- The limitations of ARIMA when it comes to seasonal data.
- The SARIMA extension of ARIMA that explicitly models the seasonal element in univariate data.
- How to implement the SARIMA method in Python using the Statsmodels library.

Kick-start your project with my new book Time Series Forecasting With Python, including *step-by-step tutorials* and the *Python source code* files for all examples.

Let's get started.

Update: For help using and grid searching SARIMA hyperparameters, see this post:

- How to Grid Search SARIMA Model Hyperparameters for Time Series Forecasting

A Gentle Introduction to SARIMA for Time Series Forecasting in Python
Photo by Mario Micklisch, some rights reserved.

Tutorial Overview

This tutorial is divided into four parts; they are:

1. What's Wrong with ARIMA
2. What Is SARIMA?
3. How to Configure SARIMA
4. How to use SARIMA in Python

What's Wrong with ARIMA

Autoregressive Integrated Moving Average, or ARIMA, is a forecasting method for univariate time series data.

As its name suggests, it supports both an autoregressive and moving average elements. The integrated element refers to differencing allowing the method to support time series data with a trend.

A problem with ARIMA is that it does not support seasonal data. That is a time series with a repeating cycle.

ARIMA expects data that is either not seasonal or has the seasonal component removed, e.g. seasonally adjusted via methods such as seasonal differencing.

For more on ARIMA, see the post:

- How to Create an ARIMA Model for Time Series Forecasting with Python

An alternative is to use SARIMA.

What is SARIMA?

Seasonal Autoregressive Integrated Moving Average, SARIMA or Seasonal ARIMA, is an extension of ARIMA that explicitly supports univariate time series data with a seasonal component.

It adds three new hyperparameters to specify the autoregression (AR), differencing (I) and moving average (MA) for the seasonal component of the series, as well as an additional parameter for the period of the seasonality.

> *A seasonal ARIMA model is formed by including additional seasonal terms in the ARIMA […] The seasonal part of the model consists of terms that are very similar to the non-seasonal components of the model, but they involve backshifts of the seasonal period.*

— Page 242, Forecasting: principles and practice, 2013.

How to Configure SARIMA

Configuring a SARIMA requires selecting hyperparameters for both the trend and seasonal elements of the series.

Trend Elements

There are three trend elements that require configuration.

They are the same as the ARIMA model; specifically:

- **p**: Trend autoregression order.
- **d**: Trend difference order.
- **q**: Trend moving average order.

Seasonal Elements

There are four seasonal elements that are not part of ARIMA that must be configured; they are:

- **P**: Seasonal autoregressive order.
- **D**: Seasonal difference order.

- **Q**: Seasonal moving average order.
- **m**: The number of time steps for a single seasonal period.

Together, the notation for an SARIMA model is specified as:

```
SARIMA(p,d,q)(P,D,Q)m
```

Where the specifically chosen hyperparameters for a model are specified; for example:

```
SARIMA(3,1,0)(1,1,0)12
```

Importantly, the *m* parameter influences the *P*, *D*, and *Q* parameters. For example, an m of 12 for monthly data suggests a yearly seasonal cycle.

A *P*=1 would make use of the first seasonally offset observation in the model, e.g. t-(m*1) or t-12. A *P*=2, would use the last two seasonally offset observations t-(m * 1), t-(m * 2).

Similarly, a *D* of 1 would calculate a first order seasonal difference and a *Q*=1 would use a first order errors in the model (e.g. moving average).

> *A seasonal ARIMA model uses differencing at a lag equal to the number of seasons (s) to remove additive seasonal effects. As with lag 1 differencing to remove a trend, the lag s differencing introduces a moving average term. The seasonal ARIMA model includes autoregressive and moving average terms at lag s.*

— Page 142, Introductory Time Series with R, 2009.

The trend elements can be chosen through careful analysis of ACF and PACF plots looking at the correlations of recent time steps (e.g. 1, 2, 3).

Similarly, ACF and PACF plots can be analyzed to specify values for the seasonal model by looking at correlation at seasonal lag time steps.

For more on interpreting ACF/PACF plots, see the post:

- A Gentle Introduction to Autocorrelation and Partial Autocorrelation

> *Seasonal ARIMA models can potentially have a large number of parameters and combinations of terms. Therefore, it is appropriate to try out a wide range of models when fitting to data and choose a best fitting model using an appropriate criterion …*

— Pages 143-144, Introductory Time Series with R, 2009.

Alternately, a grid search can be used across the trend and seasonal hyperparameters.

For more on grid searching SARIMA parameters, see the post:

- How to Grid Search SARIMA Model Hyperparameters for Time Series Forecasting

How to use SARIMA in Python

The SARIMA time series forecasting method is supported in Python via the Statsmodels library.

To use SARIMA there are three steps, they are:

1. Define the model.
2. Fit the defined model.
3. Make a prediction with the fit model.

Let's look at each step in turn.

1. Define Model

An instance of the SARIMAX class can be created by providing the training data and a host of model configuration parameters.

```
# specify training data
data = ...
# define model
model = SARIMAX(data, ...)
```

The implementation is called SARIMAX instead of SARIMA because the "X" addition to the method name means that the implementation also supports exogenous variables.

These are parallel time series variates that are not modeled directly via AR, I, or MA processes, but are made available as a weighted input to the model.

Exogenous variables are optional can be specified via the "*exog*" argument.

```
# specify training data
data = ...
# specify additional data
other_data = ...
# define model
model = SARIMAX(data, exog=other_data, ...)
```

The trend and seasonal hyperparameters are specified as 3 and 4 element tuples respectively to the "*order*" and "*seasonal_order*" arguments.

These elements must be specified.

```
# specify training data
data = ...
# define model configuration
my_order = (1, 1, 1)
my_seasonal_order = (1, 1, 1, 12)
# define model
model = SARIMAX(data, order=my_order, seasonal_order=my_seasonal_order, ...)
```

These are the main configuration elements.

There are other fine tuning parameters you may want to configure. Learn more in the full API:

- statsmodels.tsa.statespace.sarimax.SARIMAX API

2. Fit Model

Once the model is created, it can be fit on the training data.

The model is fit by calling the fit() function.

Fitting the model returns an instance of the *SARIMAXResults* class. This object contains the details of the fit, such as the data and coefficients, as well as functions that can be used to make use of the model.

```
# specify training data
data = ...
# define model
model = SARIMAX(data, order=..., seasonal_order=...)
# fit model
model_fit = model.fit()
```

Many elements of the fitting process can be configured, and it is worth reading the API to review these options once you are comfortable with the implementation.

- statsmodels.tsa.statespace.sarimax.SARIMAX.fit API

3. Make Prediction

Once fit, the model can be used to make a forecast.

A forecast can be made by calling the *forecast()* or the *predict()* functions on the *SARIMAXResults* object returned from calling fit.

The forecast() function takes a single parameter that specifies the number of out of sample time steps to forecast, or assumes a one step forecast if no arguments are provided.

```
# specify training data
data = ...
# define model
model = SARIMAX(data, order=..., seasonal_order=...)
# fit model
model_fit = model.fit()
# one step forecast
yhat = model_fit.forecast()
```

The *predict()* function requires a start and end date or index to be specified.

Additionally, if exogenous variables were provided when defining the model, they too must be provided for the forecast period to the *predict()* function.

```
# specify training data
data = ...
# define model
model = SARIMAX(data, order=..., seasonal_order=...)
# fit model
model_fit = model.fit()
# one step forecast
yhat = model_fit.predict(start=len(data), end=len(data))
```

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Posts

- How to Grid Search SARIMA Model Hyperparameters for Time Series Forecasting
- How to Create an ARIMA Model for Time Series Forecasting with Python
- How to Grid Search ARIMA Model Hyperparameters with Python
- A Gentle Introduction to Autocorrelation and Partial Autocorrelation

Books

- Chapter 8 ARIMA models, Forecasting: principles and practice, 2013.
- Chapter 7, Non-stationary Models, Introductory Time Series with R, 2009.

API

- Statsmodels Time Series Analysis by State Space Methods
- statsmodels.tsa.statespace.sarimax.SARIMAX API
- statsmodels.tsa.statespace.sarimax.SARIMAXResults API
- Statsmodels SARIMAX Notebook

Articles

- Autoregressive integrated moving average on Wikipedia

Summary

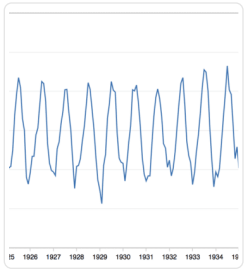
In this tutorial, you discovered the Seasonal Autoregressive Integrated Moving Average, or SARIMA, method for time series forecasting with univariate data containing trends and seasonality.

Specifically, you learned:

- The limitations of ARIMA when it comes to seasonal data.
- The SARIMA extension of ARIMA that explicitly models the seasonal element in univariate data.
- How to implement the SARIMA method in Python using the Statsmodels library.

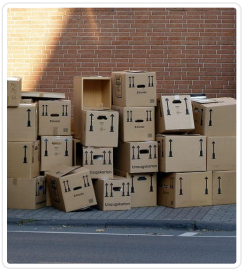
Do you have any questions?
Ask your questions in the comments below and I will do my best to answer.

More On This Topic



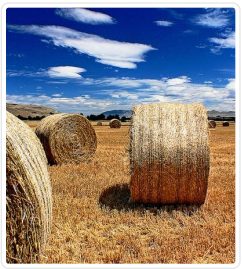
How to Grid Search SARIMA Hyperparameters for...

A Gentle Introduction to Exponential Smoothing for...

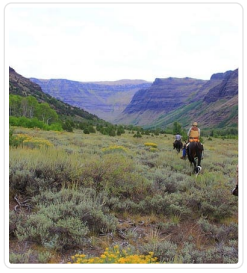


A Gentle Introduction to the Box-Jenkins Method for...

A Gentle Introduction to the Random Walk for Times...



How to Make Baseline Predictions for Time Series...



Moving Average Smoothing for Data Preparation and...

About Jason Brownlee

Jason Brownlee, PhD is a machine learning specialist who teaches developers how to get results with modern machine learning methods via hands-on tutorials.
View all posts by Jason Brownlee →